ALUMNICONNECT | DBMS PROJECT

USE CASE MODEL

Student, Alumni Mentor and Administrator Interactions

STUDENT / MENTEE	ALUMNI / MENTOR	ADMINISTRATOR
Discover mentors	Share expertise	Manage platform data
Request guidance	Manage availability	Control access
Book and review sessions	Guide and assess students	Review analytics

Prepared from the supplied "Alumni-Student Mentorship Platform: DBMS Project Blueprint"

Functional scope, UML-style use-case diagrams, detailed scenarios, business rules and DBMS alignment

Prepared for the AlumniConnect course project | 02 November 2026

1. Document purpose and scope

This document translates the supplied project blueprint into an actor-oriented use-case model. It describes what each user can do through AlumniConnect and how the platform should respond.

1.1 Product context

AlumniConnect is a mentorship, career-guidance and mock-interview platform. Students discover alumni with relevant skills and industry experience, request mentorship, reserve appointments, attend offline sessions, receive structured feedback and follow their progress. Alumni maintain professional profiles, manage availability, decide on mentorship requests and record session outcomes. Administrators maintain shared platform data and inspect platform-level analytics.

1.2 What this model covers

- Identity and access: account registration for student and mentor roles, sign-in, role-aware permissions.
- Profiles and expertise: student and mentor profile details, skills, interests, companies, departments and mentorship domains.
- Discovery and matching: mentor search, recommendations and explanations for why a mentor is relevant.
- Mentorship lifecycle: request submission, mentor decision, allocation and status history.
- Scheduling and sessions: mentor availability, booking, cancellation, offline session records and outcomes.
- Feedback and communication: session feedback, mentor reviews and event-driven notifications.
- Administration: user/role management, department/skill/domain catalogues and analytics.

1.3 Design position inherited from the blueprint

> Modular monolith with polyglot persistence
> The user-facing functions are organised as business modules in one backend application. FastAPI provides the application API boundary; MySQL 8.x with InnoDB is the system of record. Redis supports caching and temporary state. Neo4j supports relationship-based recommendations. Optional semantic/vector discovery is a later extension, not a prerequisite for the core use cases.

1.4 Reading this document

The overview diagram shows the system boundary and the major capabilities. Separate diagrams show the student, mentor and administrator portals. Detailed specifications explain triggers, preconditions, normal flows, alternatives and postconditions. The final sections connect those interactions to the project data model and DBMS concepts.

1.5 Source and interpretation

The project roles, modules, entity concepts, state transitions, workflows and technology responsibilities below are derived from the supplied project blueprint. Where the blueprint names a capability but does not define a precise policy (for example, who may cancel a confirmed booking), this document marks the policy as a project decision to configure rather than presenting it as an established rule.

2. Actors and system boundary

An actor is an external role that interacts with the system to achieve a goal. The three primary actors are the roles explicitly defined in the blueprint.

Actor	Goals and responsibilities	Typical information handled
Student / Mentee	Maintain a student profile; describe skills and interests; discover and compare mentors; submit and track requests; book appointments; attend sessions; submit feedback and mentor reviews; monitor progress.	Student profile, department, skills, domain interests, requests, bookings, session history, feedback and reviews.
Alumni / Mentor	Maintain a professional profile; describe skills, company and domain expertise; publish availability; accept or reject requests; manage appointments; conduct offline sessions; record outcomes and feedback.	Mentor profile, company, skills, expertise domains, availability, requests, allocations, bookings, session outcomes and feedback.
Administrator	Manage users and role permissions; maintain departments, skills and mentorship domains; inspect operational and aggregate analytics.	User and role records, reference catalogues, platform metrics and (where enabled) audit history.

2.1 Supporting components are not human actors

The API boundary, FastAPI modules, MySQL, Redis and Neo4j are internal implementation components. They are not additional user roles in the use-case model. MySQL owns authoritative business state; Redis data is rebuildable/temporary; Neo4j is a projection used for relationship queries. A cache hit or graph recommendation must never bypass MySQL authorization or final booking validation.

2.2 System boundary responsibilities

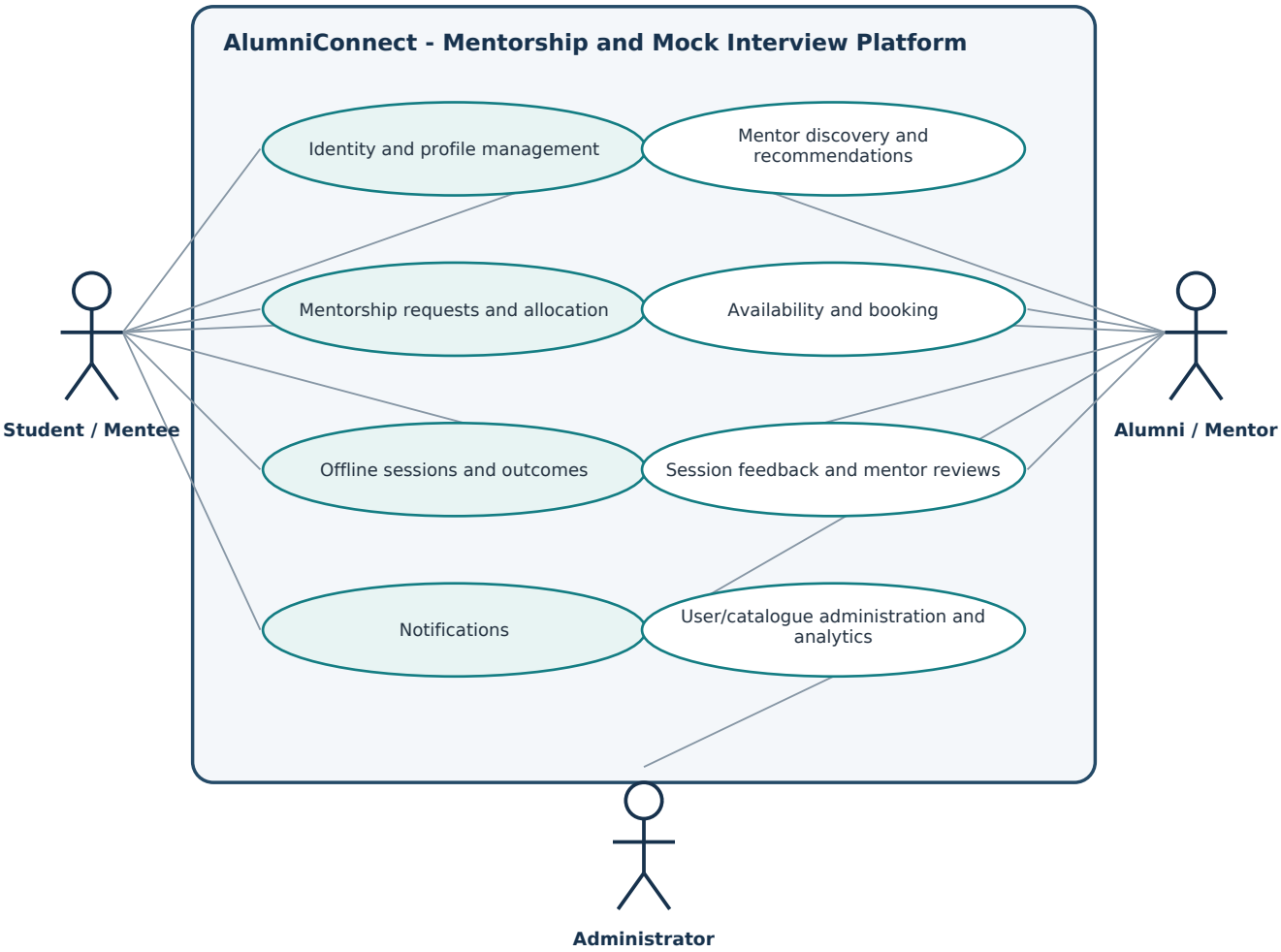
Inside AlumniConnect	Outside / not required for the core project
Registration and authentication; role-based authorization; profile and catalogue management; mentor discovery; requests and allocations; availability and booking; offline session tracking; feedback/reviews; notification event creation and delivery orchestration; analytics.	The actual offline meeting; external email/SMS delivery provider if one is configured; video-conferencing integration; payment processing; real-money transactions. The blueprint explicitly states that video conferencing is not needed for the core session workflow.

Role-based access rule
Every protected action must be authorised for the signed-in role and the relevant record. A student may act on their own requests and bookings; a mentor may act on requests assigned to them and on sessions in which they participate; an administrator may perform platform administration functions. The backend enforces these rules even when Redis or Neo4j is used.

3. High-level use-case diagram

This overview intentionally groups related functions. The following role-specific diagrams expand each group into concrete user goals.

Overview of actor participation; role-specific use cases are expanded on the next diagrams.



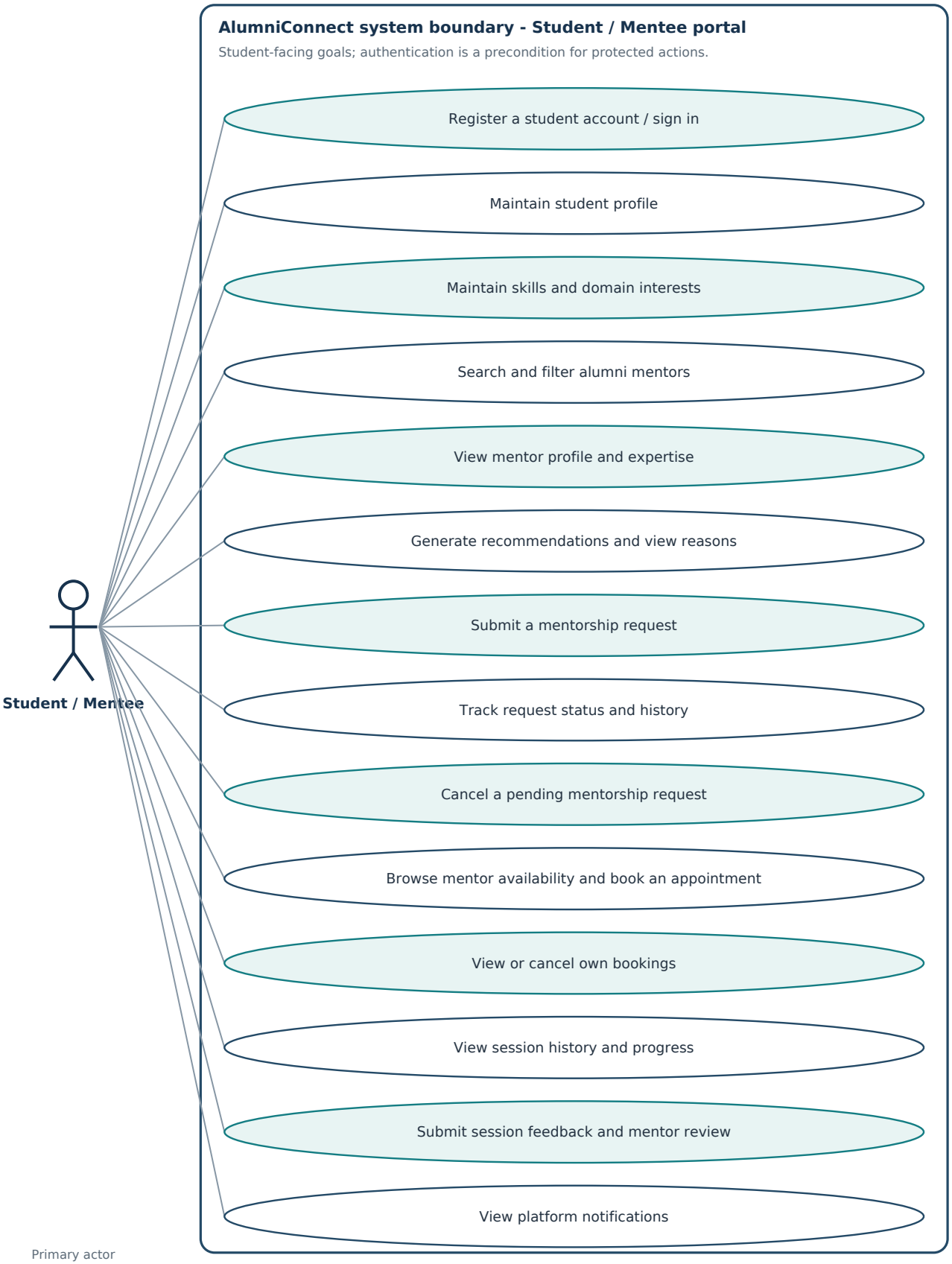
How to read the diagram: stick figures represent external actors; the rounded rectangle is the AlumniConnect system boundary; ovals represent goals supported by the platform; solid lines show actor participation. The detailed use-case catalogue later assigns stable identifiers to individual goals.

The student and mentor both participate in profile, request/booking, session and feedback capabilities, but their permissions differ. The administrator interacts with identity/catalogue administration and analytics rather than participating in a mentorship session.

> Important modelling distinction
> A recommendation is not an allocation. A request is not an allocation. A booking is a planned appointment; a mentorship session is the actual meeting and its recorded outcome. The use-case model keeps these separate because the blueprint treats them as separate business concepts.

4. Student / mentee use-case diagram

The student portal starts with the student profile and then supports a complete journey from mentor discovery to a completed, reviewed session.



5. Student use-case specifications

The following specifications focus on the student goals that drive the main platform journey.

UC-S03 / UC-S04 - Discover mentors and inspect recommendations

Field	Specification
Primary actor	Student / Mentee
Goal and trigger	Find alumni mentors whose skills, professional experience or mentorship domains align with the student's goals. Triggered when the student opens mentor discovery or requests recommendations.
Preconditions	The student is authenticated. The student profile and any available skills/interests have been saved.
Main flow	1. Student enters filters such as skill, mentorship domain or company/experience context. 2. The platform searches mentor profiles and/or generates recommendations. 3. Neo4j traverses relevant Student, Mentor, Skill, Domain and Company relationships. 4. MySQL validates authoritative eligibility and profile data. 5. The platform displays matching mentor cards and, where available, reasons for each recommendation. 6. Student opens a mentor profile for details.
Alternatives	If no exact matches exist, the platform shows an empty state and allows the student to revise filters. If recommendation data is stale, the platform refreshes the relevant projection/cache and uses MySQL as the authority.
Postconditions	The student has viewed a result list or mentor profile. Recommendation run/candidate history may be retained so the system can explain what was suggested and what the student selected.

UC-S05 - Submit a mentorship request

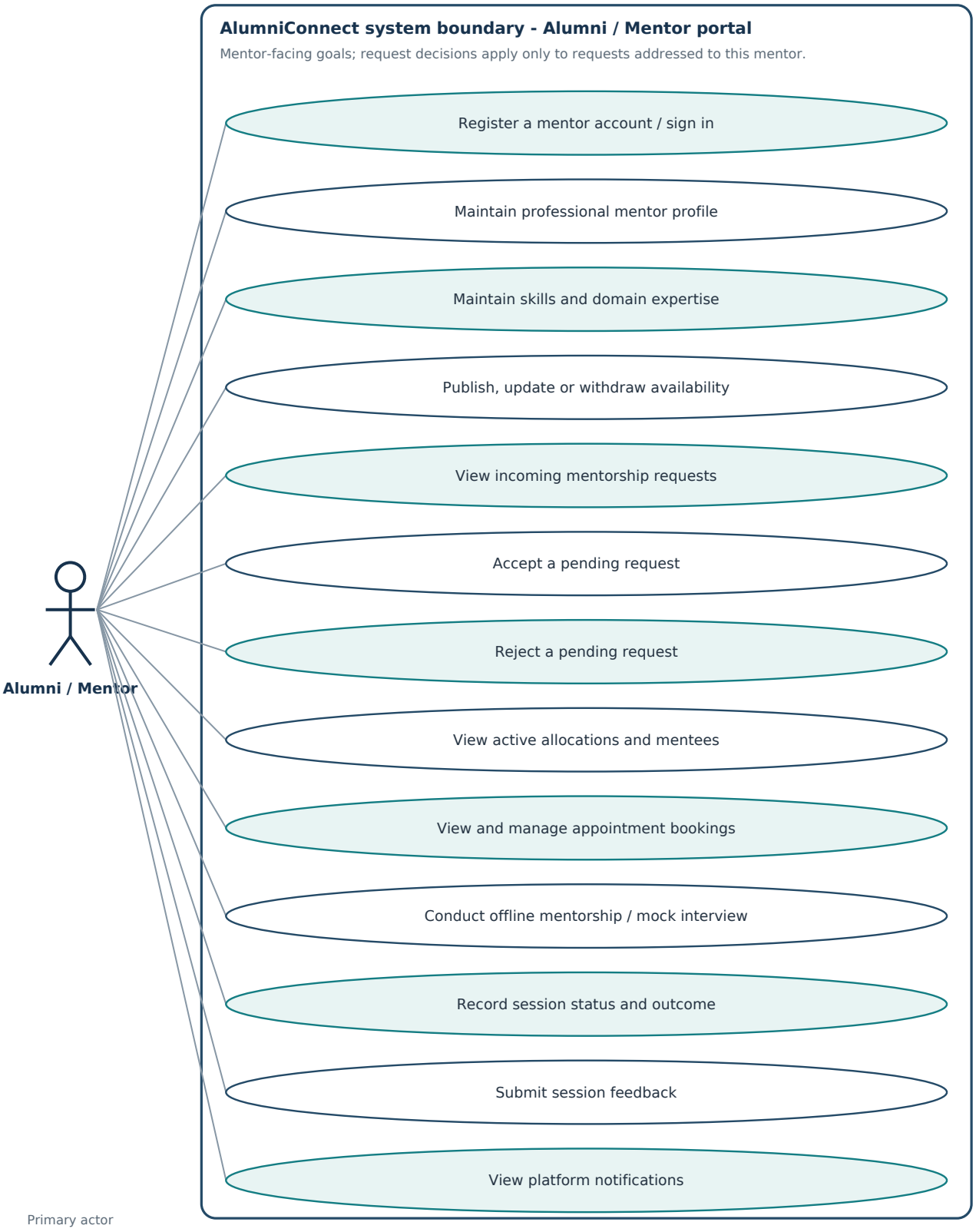
Field	Specification
Primary actor	Student / Mentee
Trigger	The student selects a mentor and chooses to request mentorship.
Preconditions	Student is authenticated; target mentor exists and is eligible to receive a request; any relevant profile details are available.
Main flow	1. Student opens the selected mentor profile. 2. Student supplies the request context or goal if the interface asks for it. 3. Platform validates the student, mentor and request state. 4. Platform creates a Mentorship Request in Pending state. 5. The request is committed in MySQL. 6. A notification event for the mentor is recorded in the same business transaction. 7. The mentor is notified asynchronously.
Alternatives	If the target is unavailable or an equivalent active relationship/request already prevents another request, the platform explains the issue and does not create a duplicate active request. If persistence fails, no request or notification event is committed.
Postconditions	A pending request exists and remains visible in the student's request history. The request is not an allocation until the mentor accepts it.

UC-S06 / UC-S07 - Track or cancel a mentorship request

The student can inspect current and historical requests, including Pending, Accepted, Rejected and Cancelled outcomes. A student may cancel a request while it is Pending. The request record is retained for history and reporting; cancellation changes its status rather than deleting it. The blueprint does not specify cancellation after acceptance, so this model does not assume that a student can unilaterally cancel an active allocation.

6. Alumni / mentor use-case diagram

The mentor portal supports professional profile management, request decisions, availability publishing and the delivery/recording of mentorship sessions.



7. Mentor use-case specifications

The mentor decision flow and availability management have direct consequences for allocation and booking integrity.

UC-M04 / UC-M05 - View, accept or reject a mentorship request

Field	Specification
Primary actor	Alumni / Mentor
Trigger	A new mentorship request is received or the mentor opens the pending-request queue.
Preconditions	Mentor is authenticated; the request belongs to this mentor; request is Pending.
Main flow - accept	1. Mentor opens request details. 2. Mentor selects Accept. 3. Platform rechecks that the request is still Pending. 4. Platform verifies that the mentor can accept the allocation. 5. Platform checks for a duplicate active allocation. 6. Platform creates one Mentor Allocation and updates the request to Accepted in one SQL transaction. 7. The transaction records a notification event. 8. Student is notified after commit.
Alternative - reject	1. Mentor selects Reject. 2. Platform verifies that the request is still Pending and belongs to the mentor. 3. Request status becomes Rejected. 4. Notification event is recorded in the same transaction. 5. Student is notified after commit. No allocation is created.
Exceptions	If another action has already changed the request state, the decision is rejected as stale. If the transaction fails, neither the status change nor the allocation is committed.
Postconditions	Accepted: exactly one active allocation is associated with the request. Rejected: request remains in history as Rejected and no allocation is created.

UC-M01 / UC-M02 - Maintain mentor profile and expertise

The mentor creates or updates professional information, company/experience details, skills and mentorship-domain expertise. Skills and domains are represented as connected records rather than repeated free-text fields. Updates are authoritative in MySQL; relevant Neo4j projections and Redis caches are updated or invalidated so future recommendations reflect the change.

UC-M03 - Publish and manage availability

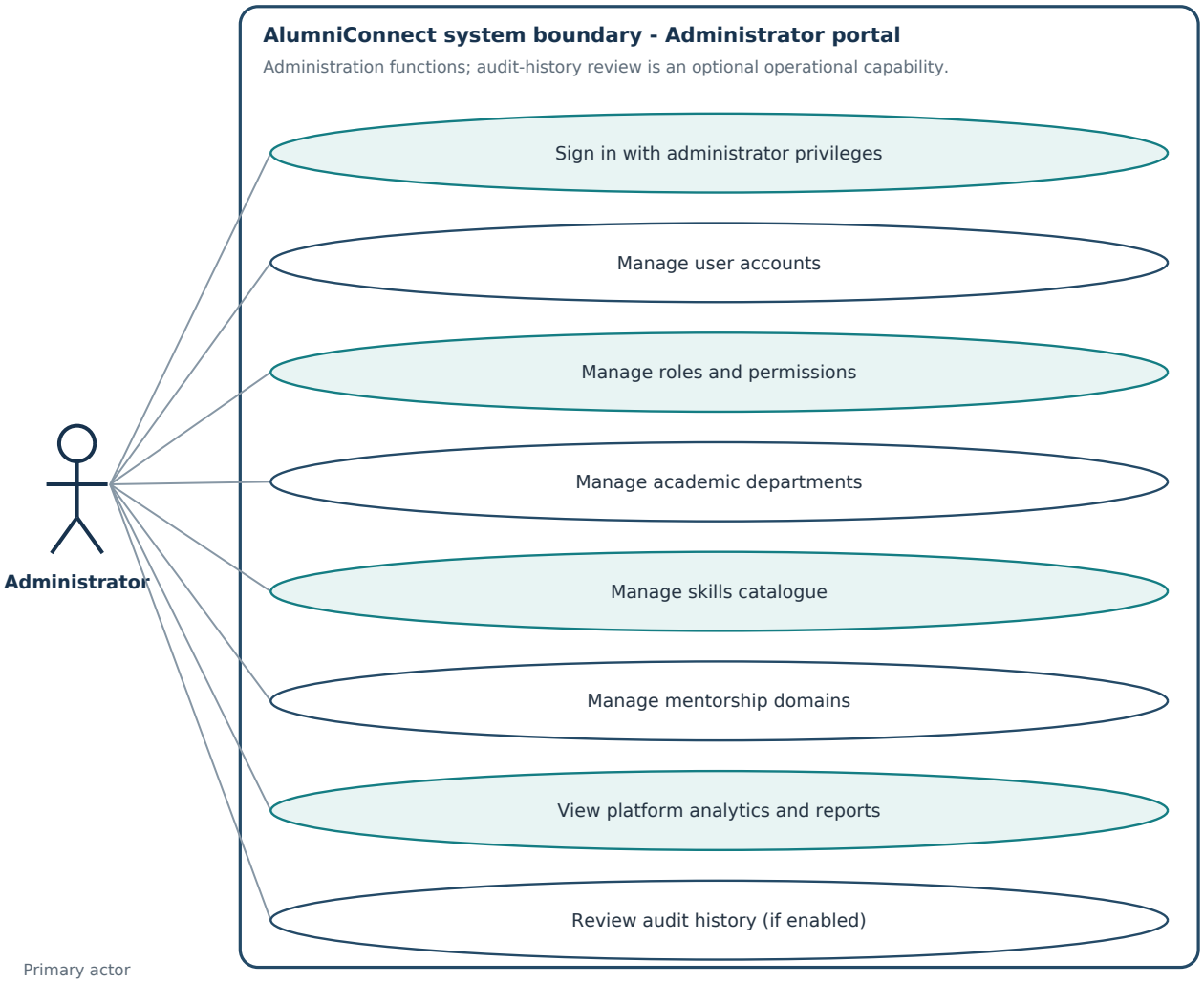
The mentor publishes one or more available time slots and may update or withdraw slots that are not already committed to a booking. The platform validates time ranges and overlapping mentor availability. A published slot is discoverable by students, but it is not guaranteed to remain available until the booking transaction commits. MySQL performs the final availability check and slot reservation.

UC-M06 / UC-M07 - View allocations and appointments

The mentor views accepted mentorship relationships and the bookings attached to those relationships. The mentor can inspect appointment details, prepare for the offline session and follow the configured cancellation workflow. Cancellation of a confirmed booking must notify both participants. The exact cancellation window and whether either participant can initiate it are policy decisions to configure.

8. Administrator use-case diagram

The administrator governs platform-wide identities, role permissions and reference catalogues, and uses reporting to understand platform activity.



Administrator interaction summary

The administrator maintains the reference data used by profile entry, search and recommendation: departments, skills and mentorship domains. The administrator also manages user accounts and role/permission assignments and reviews aggregate analytics such as the request-to-allocation-to-session funnel and mentor feedback summaries.

> Separation of administrative and mentorship authority
> The blueprint assigns platform administration and analytics to the administrator. It does not assign the administrator the right to impersonate a student/mentor, accept mentorship requests on behalf of a mentor, or edit session outcomes. Such capabilities should not be added without an explicit project requirement.

9. Administrator use-case specifications

Administrative functions maintain the shared data and access rules on which student and mentor workflows depend.

UC-A01 / UC-A02 - Manage users, roles and permissions

Field	Specification
Primary actor	Administrator
Trigger	Administrator opens the account administration area.
Preconditions	Administrator is authenticated and has an administrator role.
Main flow	1. Administrator searches or selects a user account. 2. Platform displays permitted account and role information. 3. Administrator updates the supported account attributes or role/permission assignment. 4. Platform validates uniqueness and role rules. 5. Changes are saved in MySQL and an audit record is written for significant administrative changes.
Alternatives	Invalid role combinations or duplicate identity values are rejected. A user cannot grant themselves administrator privileges through the student/mentor interface.
Postconditions	The authoritative user/role state is updated; access checks use the updated state.

UC-A03 / UC-A04 / UC-A05 - Manage departments, skills and mentorship domains

The administrator creates, updates or retires reference entries used by student and mentor profiles. These catalogues support consistent structured data, search filters, SQL reporting and graph relationships. Before a catalogue item is retired, the platform should preserve referential integrity for existing profiles and history; it should not silently cascade-delete historical requests or sessions.

UC-A06 - View analytics and reports

The administrator opens aggregate views for platform operation. The blueprint identifies request-to-allocation-to-session funnel analysis and mentor-rating reports as useful reporting goals. Example measures include requests submitted, requests accepted, allocations created, sessions completed, average mentor ratings and mentors with ratings above a threshold after a minimum number of completed sessions. Reports are read-only and must respect any privacy restrictions on personal data.

UC-A07 - Review audit history (optional)

The blueprint proposes an Audit Log entity for important historical changes. An administrative audit-history screen is a reasonable operational extension, but it is not a mandatory core actor goal in the blueprint. Where implemented, it should expose event time, actor, action, target record and outcome without allowing historical audit entries to be silently overwritten.

10. Shared use-case specification: book an appointment

Booking is one of the strongest DBMS demonstrations in this project because it requires correctness under concurrent users.

UC-S09 - Book a mentorship / mock-interview appointment

Field	Specification
Primary actor	Student / Mentee
Supporting participant	Alumni / Mentor (owns the published slot and later attends the meeting).
Goal	Reserve a mentor-published time slot for a valid mentorship relationship or request context.
Trigger	Student selects an available slot and confirms the booking.
Preconditions	Student is authenticated; mentor and slot exist; the request/allocation is valid for the booking type; slot is within a permitted booking window.
Main flow	1. Student opens the mentor availability calendar. 2. Platform displays candidate slots. 3. Student selects a slot and confirms. 4. Backend rechecks the slot against MySQL (not Redis). 5. Backend checks whether the student has another overlapping appointment. 6. Backend checks mentor capacity and the validity of the request/allocation. 7. Backend checks for a duplicate submission/idempotency key. 8. In a transaction, the backend locks/claims the slot, creates the Booking, marks the slot as no longer available and records notification events. 9. Transaction commits. 10. Both participants receive booking confirmation.
Alternative / conflict	If another student has already claimed the slot, or the student has an overlapping appointment, the transaction is rolled back and the student is asked to choose another slot. A repeated request with the same idempotency key must not create a second booking.
Postconditions	Exactly one valid booking exists for the slot; the slot cannot be claimed again; both participants can view the confirmed booking.

Concurrency and transaction requirement
The blueprint specifically calls for database uniqueness and transactional locking to prevent two requests from claiming the same mentor slot. Student-side overlapping appointments must also be protected. The booking decision is atomic: either the booking and slot reservation both commit, or neither does.

Booking and cancellation rules

- MySQL is the final authority for availability; Redis availability data may be stale and must be revalidated.
- The slot must be available at commit time, not merely when the calendar was first displayed.
- A student must not hold overlapping appointments, even when the mentor slots are different.
- Mentor capacity and the relationship/request validity must be checked before confirming.
- A confirmed booking is a plan. It does not prove that a session took place.
- The blueprint includes booking-cancelled notifications to both participants. Cancellation initiator, deadline and rescheduling policy remain configurable business decisions.

11. Session outcome, feedback and notifications

The actual offline event is modelled separately from its planned booking.

UC-M08 / UC-M09 - Conduct and record an offline session

Primary actor: Alumni / Mentor. Trigger: the scheduled offline meeting has taken place or its outcome is known. Precondition: a valid Booking exists and the session participants are identified.

Main flow: (1) Mentor opens the scheduled appointment. (2) Mentor records whether the session was Completed, Cancelled or No-show. (3) For a completed session, mentor records discussion topics, skills assessed, performance observations, areas for improvement, next steps and recommended preparation before the next session. (4) Platform validates the participant and state transition. (5) Session outcome is saved and history is preserved.

Postcondition: The Mentorship Session reflects the actual event and its outcome. A session can be Scheduled, then transition to Completed, Cancelled or No-show. The booking remains a historical record of the planned appointment.

UC-X03 / UC-S12 - Submit session feedback and mentor review

Feedback concept	Actor and rules
Session feedback	Either actual participant may submit structured feedback about communication, helpfulness, technical preparation and session notes. A session cannot receive feedback before it has taken place. Only the actual participants may submit it.
Mentor review	The student may submit a rating/review of the mentor after a completed session. At most one student mentor review is allowed per session. The review must reference a valid completed session. Review history should not be silently overwritten.

UC-X01 - View notifications (all roles)

Students and mentors can view notifications relevant to their account. The blueprint defines the following event-to-recipient mapping:

Business event	Recipient
Mentorship request submitted	Mentor
Request accepted / rejected	Student
Booking confirmed / cancelled	Both participants
Session reminder	Both participants
Feedback submitted	Relevant participant

> Reliable notification processing
> The notification event is recorded in MySQL within the same business transaction as the operation. A background worker delivers it after commit. This transactional outbox approach prevents a message from being sent for a request/booking change that later rolls back.

12. Consolidated use-case catalogue

Stable identifiers make the model easier to map to implementation tasks, API endpoints, test cases and project demonstrations.

ID	Use case	Primary actor(s)	Priority
UC-C01	Register student or mentor account	Student; Mentor	Core
UC-C02	Authenticate and sign out	All three roles	Core
UC-S01	Maintain student profile	Student	Core
UC-S02	Maintain student skills and domain interests	Student	Core
UC-S03	Search mentors and view recommendations	Student	Core
UC-S04	View mentor profile and recommendation reasons	Student	Core
UC-S05	Submit mentorship request	Student	Core
UC-S06	Track request status and history	Student	Core
UC-S07	Cancel a pending request	Student	Core
UC-M01	Maintain mentor professional profile	Mentor	Core
UC-M02	Maintain mentor skills and domain expertise	Mentor	Core
UC-M03	Publish/update/withdraw availability	Mentor	Core
UC-M04	View incoming mentorship requests	Mentor	Core
UC-M05	Accept or reject a pending request	Mentor	Core
UC-S08	View active mentorship allocation	Student	Core
UC-M06	View active allocations and mentees	Mentor	Core

12. Consolidated use-case catalogue (continued)

ID	Use case	Primary actor(s)	Priority
UC-S09	Browse availability and book appointment	Student	Core
UC-S10	View own bookings	Student	Core
UC-X02	Cancel a confirmed booking	Student or Mentor*	Core workflow
UC-M07	View/manage appointment bookings	Mentor	Core
UC-M08	Conduct offline mentorship/mock interview	Mentor; Student participates	Core
UC-M09	Record session status and outcome	Mentor	Core
UC-S11	View session history and progress	Student	Core
UC-X03	Submit structured session feedback	Student; Mentor	Core
UC-S12	Submit mentor rating/review	Student	Core
UC-X01	View relevant notifications	Student; Mentor; Admin	Core
UC-A01	Manage user accounts	Administrator	Core
UC-A02	Manage roles and permissions	Administrator	Core
UC-A03	Manage departments	Administrator	Core
UC-A04	Manage skills catalogue	Administrator	Core
UC-A05	Manage mentorship domains	Administrator	Core
UC-A06	View platform analytics and reports	Administrator	Core
UC-A07	Review audit history	Administrator	Optional

*The blueprint requires a booking-cancelled notification to both participants but does not explicitly define who may initiate cancellation or its policy window. Treat this as a configurable policy decision.

13. UML relationships and lifecycle rules

Use <> when a base use case always invokes a required sub-function. Use <> only for optional or conditional behaviour added to a base use case. Preconditions are not automatically include relationships.

13.1 Recommended <> relationships

Base use case	Included use case	Reason
Accept mentorship request	Validate request is Pending	The decision must be rechecked at the time of acceptance.
Accept mentorship request	Verify mentor capacity / eligibility	Prevents invalid or duplicate active allocations.
Accept mentorship request	Create allocation and update request	These changes form one logical business operation.
Book appointment	Revalidate selected slot	The slot may have been claimed after the calendar was displayed.
Book appointment	Check student overlap and mentor capacity	Both sides of the scheduling constraint must be enforced.
Book appointment	Commit booking and slot reservation	The reservation is atomic and protected against concurrent claims.
Submit important request/booking change	Record notification outbox event	A notification must correspond to a committed business event.

13.2 Recommended <> relationships

There is no mandatory optional extension required by the blueprint. A future implementation could model an optional "view recommendation explanation" extension on "view recommendations" or a future vector-semantic search as an extension to mentor discovery. Keep vector search outside the core release because the blueprint explicitly places it in Phase 2.

13.3 Request, booking and session state models

Object	Allowed lifecycle from the blueprint	Interpretation
Mentorship Request	Pending -> Accepted / Rejected / Cancelled	Accepted may produce one allocation. Rejected and cancelled requests remain in history.
Booking	Planned/confirmed -> Cancelled (where cancellation is permitted)	A booking records the planned appointment. Cancellation policy is configurable.
Mentorship Session	Scheduled -> Completed / Cancelled / No-show	The mentor records what actually happened; preserve session history.
Mentor Allocation	Created after accepted request	A request can produce at most one allocation; active duplicates are prevented.

14. Use cases mapped to the project data and databases

This mapping shows where use-case information is authoritative and which specialised stores support it.

14.1 Conceptual entities involved

Domain	Blueprint entities / records	Use-case purpose
Identity and profiles	User, Role, Student Profile, Mentor/Alumni Profile, Department, Company	Identify actors, enforce access and represent student/mentor context.
Skills and domains	Skill, Student Skill, Mentor Skill, Mentorship Domain, Student Domain Interest, Mentor Domain Expertise	Represent many-to-many skills and areas of interest/expertise used in discovery.
Discovery and allocation	Mentorship Request, Recommendation Run, Recommended Mentor, Mentor Allocation	Separate a suggested candidate from a request and an accepted relationship.
Scheduling and sessions	Mentor Availability, Booking, Mentorship Session	Separate available time, planned appointment and actual offline session.
Feedback and communication	Session Feedback, Mentor Review, Notification, Notification Outbox, Audit Log	Capture participant feedback, student review, reliable event delivery and history.

14.2 Storage responsibility

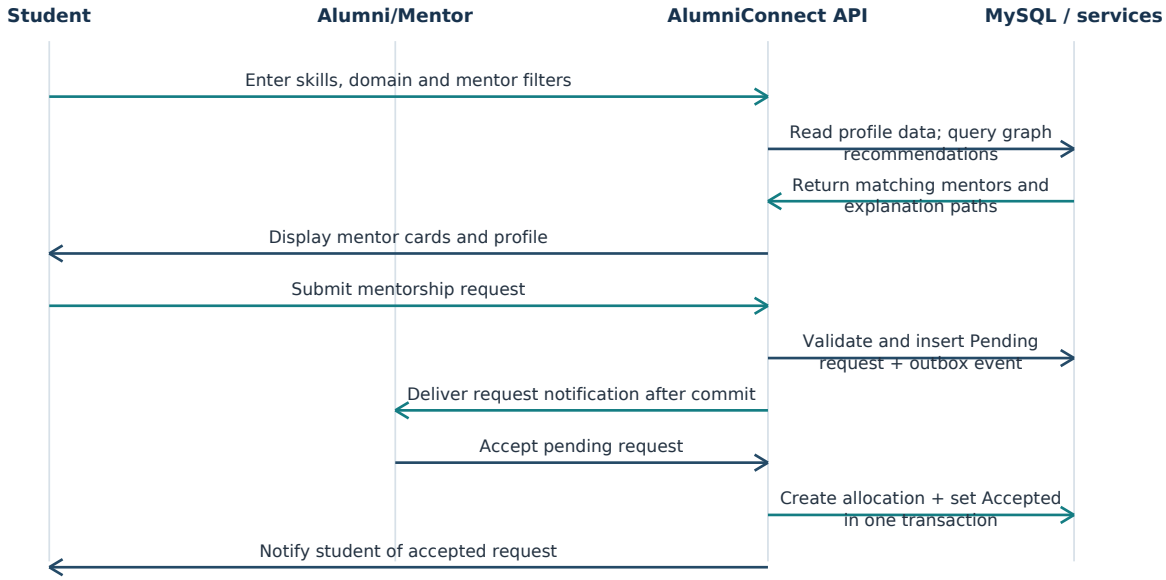
Store	Authoritative / supporting role	Use-case implication
MySQL 8.x + InnoDB	System of record for identities, profiles, skills, requests, allocations, availability, bookings, sessions, feedback, reviews, notifications and audit records.	Every permission-sensitive decision, allocation and final booking confirmation is checked against MySQL. ACID transactions protect multi-record changes.
Redis	Performance layer for frequently viewed mentor cards, popular skills/domains, recommendation results, short-lived login/session state, rate limits and idempotency keys.	Cache data is temporary/rebuildable. A cache hit cannot bypass authorization or booking validation.
Neo4j	Graph projection of students, mentors, skills, domains, companies and their relationships. Used for skill/domain/industry recommendations and explainable paths.	Recommendations use graph traversal, but MySQL determines authoritative eligibility and business state.
Redis Stack vector search (optional)	Semantic text retrieval over profile/experience descriptions; proposed as a Phase 2 extension.	Not required for the core use cases; combine its candidates with graph discovery and MySQL eligibility checks if implemented.

> Do not model database engines as actors
> Students, mentors and administrators interact with the AlumniConnect application. The application coordinates MySQL, Redis and Neo4j internally. These stores are architectural collaborators, not human use-case actors.

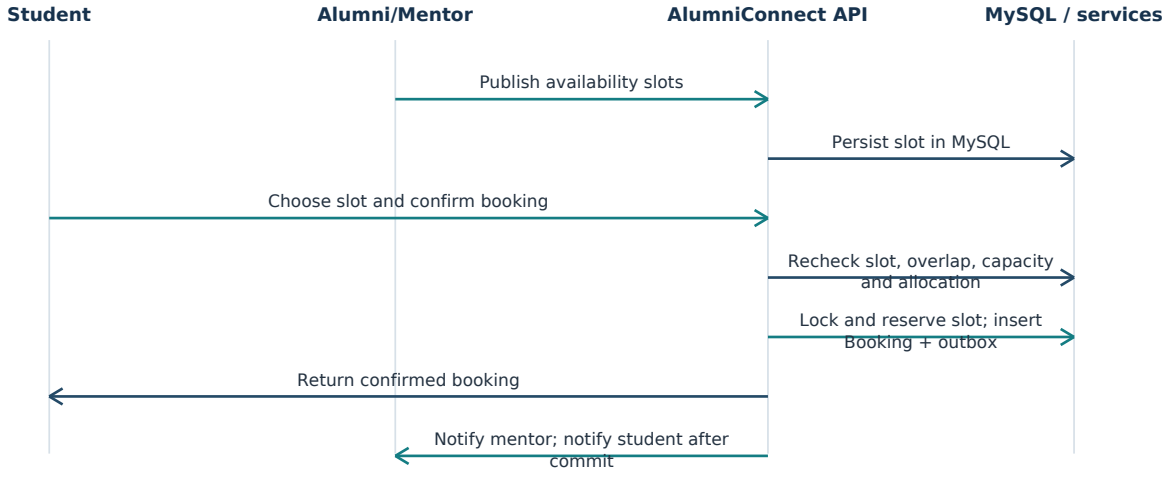
15. End-to-end interaction scenarios

These simplified interaction traces make the responsibility split between actors, the API and the data layer explicit.

15.1 Mentor discovery to accepted allocation

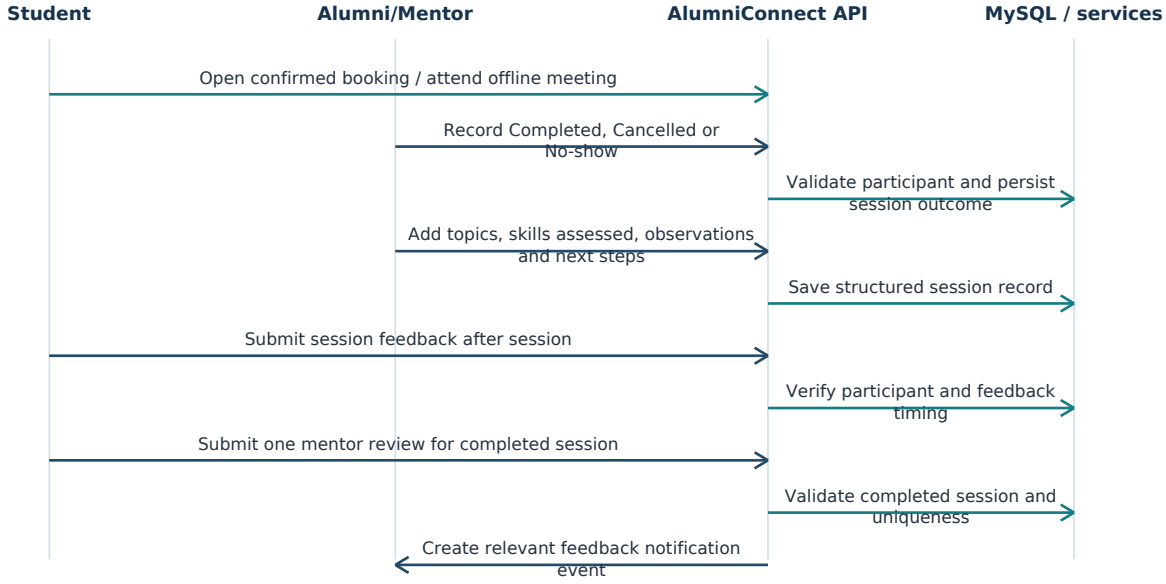


15.2 Booking a mock interview



15. End-to-end interaction scenarios (continued)

15.3 Offline session to feedback and review



15.4 Administrator catalogue and analytics scenario

The administrator signs in, updates a department/skill/domain catalogue entry or a user/role assignment, and the platform validates and persists the change in MySQL. For significant changes, an audit record is appended. When the administrator opens analytics, the backend runs read-only aggregate queries over authoritative records and presents request, allocation, session and rating measures.

15.5 Transaction boundaries to demonstrate

Business operation	Records that must succeed or fail together
Accept mentorship request	Request status update + Mentor Allocation creation + notification outbox event.
Reject mentorship request	Request status update + notification outbox event.
Book appointment	Availability reservation + Booking creation + notification outbox event.
Cancel appointment	Booking cancellation state + notification outbox event.
Submit mentor review	Completed-session validation + unique review insert.

16. Business rules and acceptance checks

The following rules can be converted directly into implementation validations and functional test cases.

Rule	Expected system behaviour / acceptance check
BR-01 - Role authorization	A user can invoke only operations permitted for their role and relevant record ownership.
BR-02 - Request lifecycle	A request begins Pending and can move to Accepted, Rejected or Cancelled. A stale request decision is rejected.
BR-03 - Allocation uniqueness	An accepted request creates at most one allocation; duplicate active allocations are prevented.
BR-04 - History preservation	Rejected and cancelled requests remain queryable for reporting; session history is preserved.
BR-05 - Slot uniqueness	Two concurrent students attempting to reserve the same mentor slot cannot both succeed.
BR-06 - Student overlap	A student cannot obtain overlapping appointments even if the mentor slots are different.
BR-07 - Mentor capacity	Booking is refused if mentor capacity has been reached.
BR-08 - Authoritative availability	Redis may cache display data, but MySQL makes the final booking decision.
BR-09 - Booking idempotency	A duplicate submission does not create two bookings for the same intent.
BR-10 - Booking/session separation	A booking is a plan; a separate session record captures whether the meeting happened and its outcome.
BR-11 - Session state	A scheduled session can become Completed, Cancelled or No-show; the change is attributable to a participant/actor.
BR-12 - Feedback timing	Feedback cannot be submitted before the session has taken place.
BR-13 - Feedback authorization	Only actual session participants can submit session feedback.
BR-14 - Mentor review uniqueness	At most one student mentor review exists per completed session; review history is not silently overwritten.
BR-15 - Notification consistency	Notification events are written within the transaction and delivered after commit.
BR-16 - Graph/cache authority	Graph and cache records are derived/supporting data; MySQL remains the source of truth.

17. Requirements traceability and scope boundaries

This final mapping connects the use-case model to the project modules, course learning and implementation priorities.

Blueprint module	Representative use cases	Primary DBMS demonstration
Identity and Access	Register, sign in, manage users/roles	Primary/foreign keys, uniqueness, role checks, authorization.
Profile Management	Maintain student and mentor profiles	Relational modelling, constraints and profile views.
Skill and Domain Management	Maintain skills, interests and expertise; admin catalogues	Many-to-many junctions, joins, normalization and reference integrity.
Mentor Discovery	Search, recommendations and explanation	SQL filters/joins, Neo4j traversal, optional full-text/vector search.
Mentorship Requests	Submit, accept, reject, cancel, track history	State transitions, procedures, transactions, audit/history.
Availability and Booking	Publish slots, book and cancel appointments	Concurrency control, uniqueness, locking, ACID and idempotency.
Session Management	Record completed/cancelled/no-show and outcomes	Referential integrity, status constraints and history.
Feedback and Reviews	Submit structured feedback and mentor rating	Foreign keys, unique constraints and aggregate queries.
Notifications	View and deliver event notifications	Transactional outbox and background delivery.
Administration and Analytics	Manage catalogues/accounts; read reports	Views, CTEs, GROUP BY/HAVING, window functions and aggregate reports.

17.1 Core release versus optional extensions

Core scope (blueprint Level 1)	Optional / later scope
Student, alumni and administrator portals; role-based access; profiles; skills/domains; Neo4j mentor recommendations; requests and allocation; availability and booking; offline session tracking; feedback and ratings; notifications; SQL analytics; Redis caching.	Semantic/vector search using Redis Stack; richer recommendation experimentation; extended audit-history administration; external delivery integrations beyond the selected notification channel; online meeting/video features (not required for core).

17.2 Explicit assumptions and policy decisions

- Registration is self-service for students and mentors. Administrator account provisioning can be handled separately by the project team; the blueprint does not detail administrator onboarding.
- The student may cancel a Pending mentorship request. Post-acceptance allocation termination is not specified and is therefore not modelled as a default student action.
- Booking cancellation is represented because the blueprint requires cancellation notifications, but its initiator, deadline and rescheduling policy must be selected by the project team.
- The core session is offline. The platform records outcomes; it does not host the actual interview or video call.
- Vector/semantic discovery is Phase 2. It should not block the core SQL and Neo4j recommendation journey.

Implementation order recommended by the model
Implement identity and role checks first; then profiles/catalogues and mentor discovery; then request acceptance/allocation; then availability and transactional booking; then session recording, feedback and notifications; finally administration analytics and optional extensions.

End of Use Case Model